

Fashion Trend Intelligence Pipeline

Merging Social Signals with Sell Through Data
to Generate Actionable Trend Intelligence for Fashion Brands

By: Pratik Dongre

Built with assistance from Claude

Project Repository

GITHUB: https://github.com/cosmic-ash/confluxe_fashion_trend_analyzer

Project Structure:

FashionTrendAnalyzer/

├─ notebooks/

- | └─ 01_data_acquisition.ipynb
- | └─ 02_sell_through_analytics.ipynb
- | └─ 03_social_signal_processing.ipynb
- | └─ 04_trend_fusion_engine.ipynb
- | └─ 05_dashboard_and_insights.ipynb

├─ src/

- | └─ config.py — Centralized paths, model params, scoring weights
- | └─ data_loader.py — Data loading + schema validation
- | └─ sell_through.py — Velocity, growth, revenue metrics
- | └─ models.py — RoBERTa (LoRA), BART zero-shot, MiniLM embeddings
- | └─ social_signals.py — Signal aggregation per category
- | └─ google_trends.py — pytrends wrapper + momentum scoring
- | └─ trend_scorer.py — Composite scoring + quadrant classification
- | └─ viz.py — Plotly dashboard components

├─ models/fashion_sentiment/ LoRA adapter weights (generated by NB03)

├─ data/

- | └─ raw/ Source CSVs (H&M, Reviews)
- | └─ processed/ Intermediate parquets + checkpoints

├─ docs/ MVP + Roadmap

└─ requirements.txt

Table of Contents

1. Problem Statement
2. Approach: Pipeline Architecture
3. Data Sources
4. ML Models Used
5. Sell-Through Analytics Pipeline
6. Social Signal Processing Pipeline
7. Trend Fusion Engine
8. Dashboard & Outputs
9. Key Findings & Recommendations
10. Methodology Transparency
11. Assumptions & Limitations
12. Next Steps

1. Problem Statement

A Direct-to-Consumer fashion brand is planning its next production run and needs to decide which trending styles to over-index on. The core challenge is that fashion trends emerge from two very different worlds:

- **Structured sell-through data:** What is actually selling (transaction volumes, revenue, velocity)
- **Unstructured social signals:** What people are talking about, searching for, and getting excited about online

A style that is selling well AND generating social buzz is a strong candidate for increased production. A style with high social interest but low current sales may be an emerging opportunity. This pipeline merges both signals to generate a single, actionable Trend Intelligence Score for every product category.

The Business Question

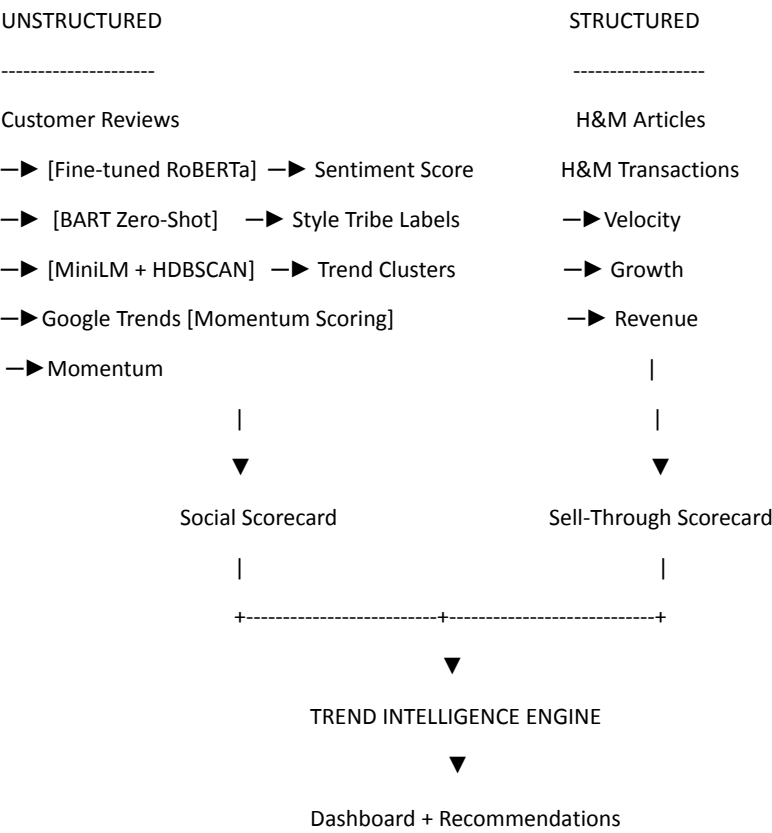
"Which trending styles should we over-index on for our next production run?"

2. Our Approach: Pipeline Architecture

The pipeline follows a five stage architecture, progressing from raw data acquisition to an interactive dashboard with actionable recommendations:

Stage	Notebook	Purpose
1. Data Acquisition	01_data_acquisition.ipynb	Download, load, validate, and profile all datasets
2. Sell-Through Analytics	02_sell_through_analytics.ipynb	Compute velocity, growth, revenue metrics from H&M transaction data
3. Social Signal Processing	03_social_signal_processing.ipynb	Run 3 ML models + Google Trends to extract social signals from reviews
4. Trend Fusion Engine	04_trend_fusion_engine.ipynb	Merge both scorecards, compute composite scores, classify into quadrants
5. Dashboard & Insights	05_dashboard_and_insights.ipynb	Interactive Plotly dashboard with executive summary and recommendations

Architecture Flow



3. Data Sources

All data sources are freely available.

Dataset	Source	Size	Type	Role in Pipeline
H&M Articles	Kaggle (Competition)	105K products	Structured	Product attributes: type, color, section, garment group, department
H&M Transactions	Kaggle (Competition)	31M+ records	Structured	Sell-through data: date, price, customer_id, article_id
Women's Clothing Reviews	Kaggle (Dataset)	23K+ reviews	Unstructured	Review text, ratings, department, class, recommendation flag
Google Trends	pytrends API	27 keywords	Semi-structured	Search interest over time for fashion style keywords

Why These Sources Work

- **H&M transactions:** Real-world sell-through data at scale (31M+ records over 2 years) represents actual consumer purchasing behavior
- **Women's Clothing Reviews:** Rich unstructured text with sentiment, style language, and product category labels; serves as a proxy for social media signals
- **Google Trends:** Real-time search interest data, captures what consumers are actively looking for right now

4. ML Models Used

Three pre-trained transformer models are applied to extract signals that rule-based methods (like VADER or keyword matching) cannot capture:

Pipeline A: Fine-Tuned Fashion Sentiment (LoRA)

Attribute	Detail
Base Model	cardiffnlp/twitter-roberta-base-sentiment-latest (~125M params)
Fine-Tuning Method	LoRA via PEFT library (rank=16, alpha=32)
Trainable Parameters	~0.5% of total (efficient adaptation)
Training Data	23K+ fashion reviews with rating derived sentiment labels
Training Time	~15 minutes on T4 GPU
Why Not VADER?	VADER misses fashion-domain nuance ('flattering' = positive, 'runs small' = negative in context)
Evaluation	Side-by-side comparison: Fine-tuned RoBERTa vs Base RoBERTa vs VADER on held-out test set

Pipeline B: Zero-Shot Style Tribe Classification

Attribute	Detail
Model	facebook/bart-large-mnli (~400M params)
Training Required	None inference only (zero-shot)
Style Tribes	minimalist, streetwear, bohemian, preppy, athleisure, quiet luxury, Y2K revival, oversized comfort, bold prints, sustainable basics
Key Insight	Creates a trend-style dimension that does NOT exist in the raw data
Output	Per-review style label + confidence score

Pipeline C: Semantic Trend Clustering

Attribute	Detail
Model	sentence-transformers/all-MiniLM-L6-v2 (~23M params)
Method	Embed → UMAP dimensionality reduction → HDBSCAN clustering
Purpose	Discover emergent trend themes beyond predefined labels
Example Discoveries	Clusters like 'wedding guest cocktail', 'postpartum comfort', 'office-to-dinner'
Visualization	Interactive UMAP scatter plot colored by cluster

Why Three Models?

Each model captures a different dimension of the social signal:

- Sentiment tells us HOW people feel about a style (enthusiasm level)
- Style tribes tell us WHAT aesthetic category a style belongs to
- Clusters discover EMERGENT themes that predefined categories miss

Together, they produce a far richer social signal than any single approach.

5. Sell-Through Analytics Pipeline

The structured side of the pipeline transforms H&M transactions into a scorecard at the (product_type, colour_group) level.

Core Metrics

Metric	Definition	Why It Matters
Sell-Through Velocity	Average units sold per week per product category	Measures current demand strength
Revenue Intensity	Revenue per SKU per week	Separates high-value from low-value volume
Week-over-Week Growth	Sales acceleration/deceleration over trailing 8 weeks	Identifies trend direction (rising vs falling)
Composite Sell-Through Score	Weighted blend: 50% velocity + 30% revenue + 20% growth	Single normalized score for downstream fusion

Classification

Class	Criteria	Business Meaning
Rising Star	High velocity + positive growth	Trending upward; increase allocation
Fading Favorite	High velocity + negative growth	Still selling but declining; hold steady
Emerging	Low velocity + positive growth	Early signal; worth watching
Dormant	Low velocity + negative growth	Underperforming; consider deprioritizing

6. Social Signal Processing Pipeline

Four parallel pipelines extract trend signals from unstructured data, producing a unified Social Signal Scorecard at the category level:

Pipeline	Model / Source	Input	Output
A: Sentiment	Fine-tuned RoBERTa (LoRA)	23K+ review texts	Sentiment score per category (-1 to +1)
B: Style Tribes	BART zero-shot (MNLI)	23K+ review texts	Style tribe distribution per category
C: Clusters	MiniLM + UMAP + HDBSCAN	23K+ review texts	Emergent trend clusters + theme labels
D: Trends	Google Trends (pytrends)	27 fashion keywords	Trend momentum + breakout flags per category

The four pipelines are aggregated into a single `social_signal_scorecard.parquet`.
One row per category with `sentiment_normalized`, `style_buzz_normalized`, `cluster_strength_normalized`, and `trend_momentum_normalized`.

7. Trend Fusion Engine

The core intelligence layer merges the sell-through scorecard with the social signal scorecard via a common category taxonomy, then computes a composite score.

Composite Trend Intelligence Score

Signal	Weight	Source
Sell-Through Velocity	25%	H&M transactions
Sentiment Lift	25%	Fine-tuned RoBERTa (Pipeline A)
Trend Momentum	20%	Google Trends (Pipeline D)
Style Buzz	15%	BART zero-shot (Pipeline B)
Cluster Strength	15%	MiniLM + HDBSCAN (Pipeline C)

All signals are min-max normalized to [0, 1] before weighting. Weights are configurable in `src/config.py` and can be tuned via backtesting.

Quadrant Classification

Quadrant	Social Signal	Sell-Through	Recommended Action
HOT	Above Median	Above Median	Over-index aggressively strong demand + rising social buzz
EMERGING	Above Median	Below Median	Test & invest high social interest, sales haven't caught up yet
FADING	Below Median	Above Median	Maintain current levels selling well but social interest waning
COLD	Below Median	Below Median	Deprioritize low demand and weak social signals

8. Dashboard & Outputs

Notebook 05 produces an interactive Plotly dashboard, the deliverable a designer or merchandiser would interact with. Key components:

Component	Chart Type	What It Shows
Trend Quadrant (Hero Chart)	Scatter plot	x=Social Score, y=Sell-Through Score, size=Revenue, color=Quadrant, the definitive 'what to over-index on' view
Top-N Recommendations	Interactive table	Ranked list of categories with confidence scores and plain-language recommendations
Style-Color Heatmap	Heatmap	Product type × Color group, colored by composite trend score
Trend Momentum Time Series	Line chart	Google Trends interest over time for top fashion keywords
Category Treemap	Treemap	Hierarchical view: Quadrant → Category, sized by trend score
Signal Radar	Radar chart	Multi-dimensional view showing how each of the 5 signals contributes for top categories
Methodology Waterfall	Waterfall chart	Score decomposition: how each signal contributes to a category's final score

8.1 Trend Quadrant View

Trend Intelligence Quadrant: What to Over-Index On



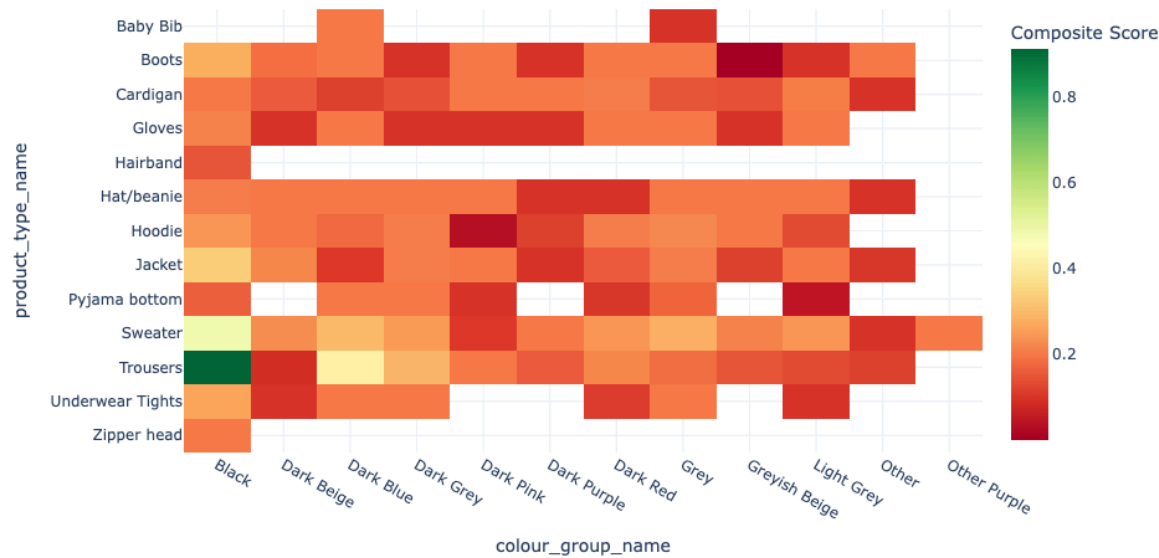
8.2 Top N Recommendations

Top Trend Recommendations — Styles to Over-Index On

Rank	Category	Quadrant	Score	Confidence	Dominant Style	Recommendation
1	Other	EMERGING	0.56	56.0%	oversized comfort	Test & invest — high social buzz, sales haven't caught up yet
2	Shorts	HOT	0.495	49.5%	quiet luxury	Over-index aggressively — strong demand + rising social signals
3	Prints & Patterns	HOT	0.4	40.0%	null	Over-index aggressively — strong demand + rising social signals
4	Shoes	EMERGING	0.398	39.8%	null	Test & invest — high

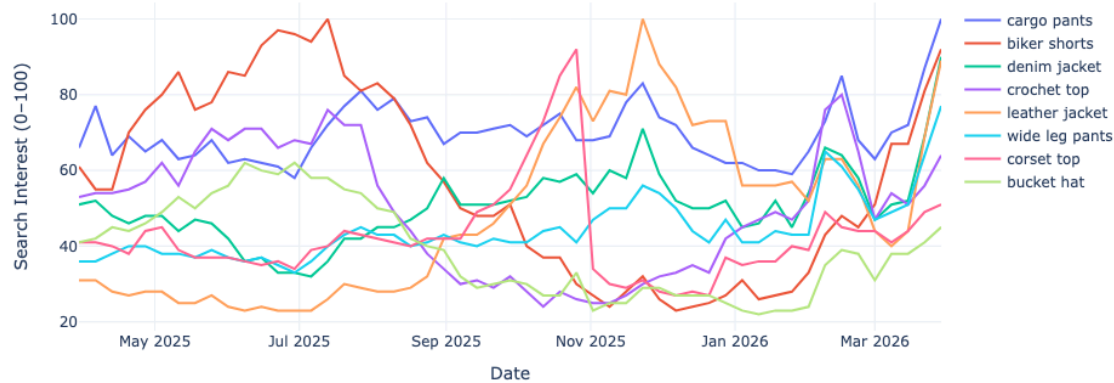
8.3 Style- Colour Heatmap

Style x Color Sell-Through Heatmap



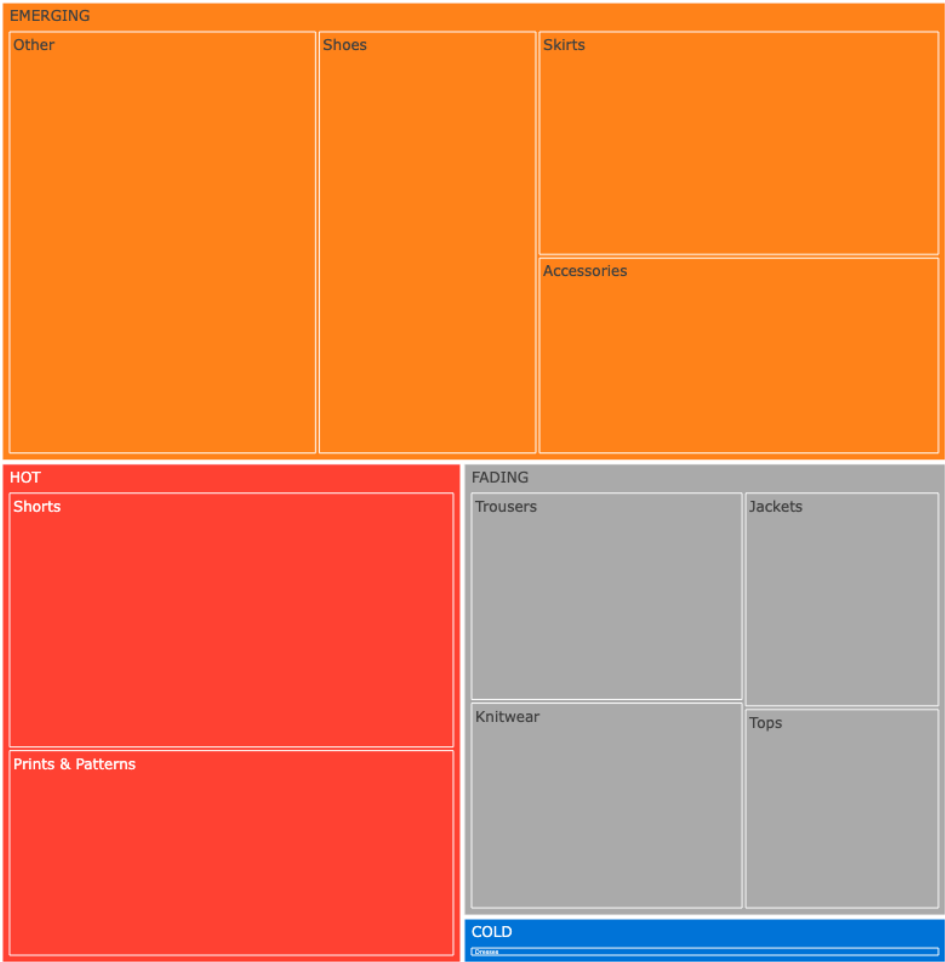
8.4 Trend Momentum Time Series

Fashion Trend Momentum — Google Trends Interest Over Time



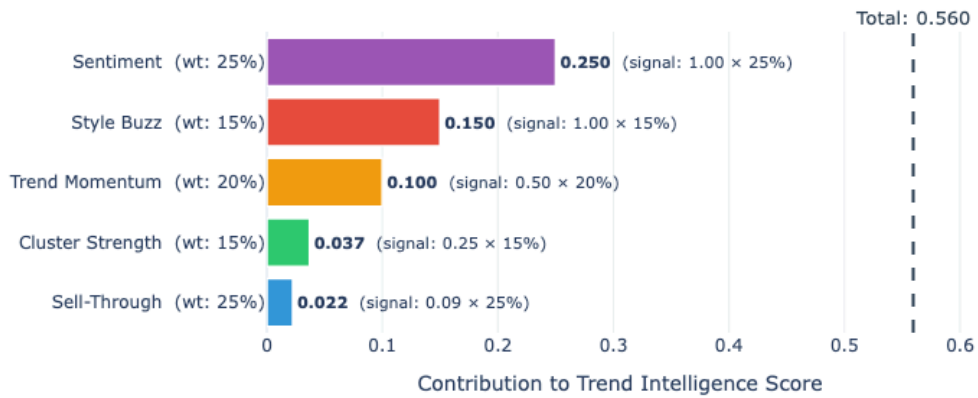
8.5 Category Treemap

Category Drill-Down by Trend Intelligence Score

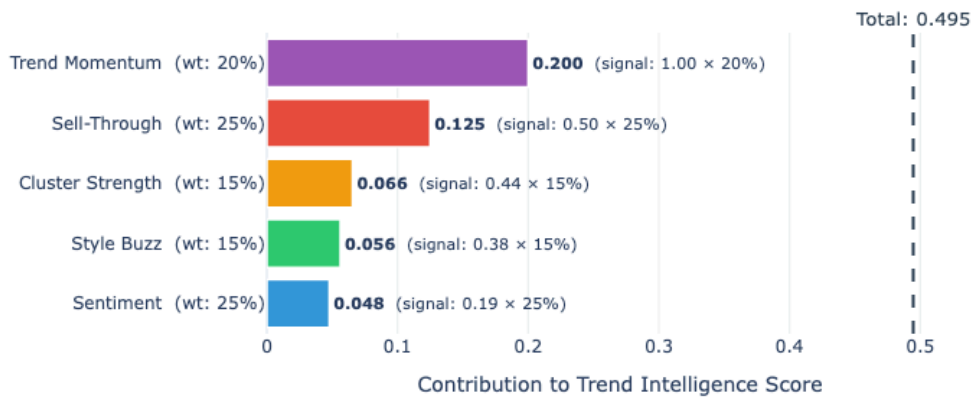


8.6 Methodology Waterfall

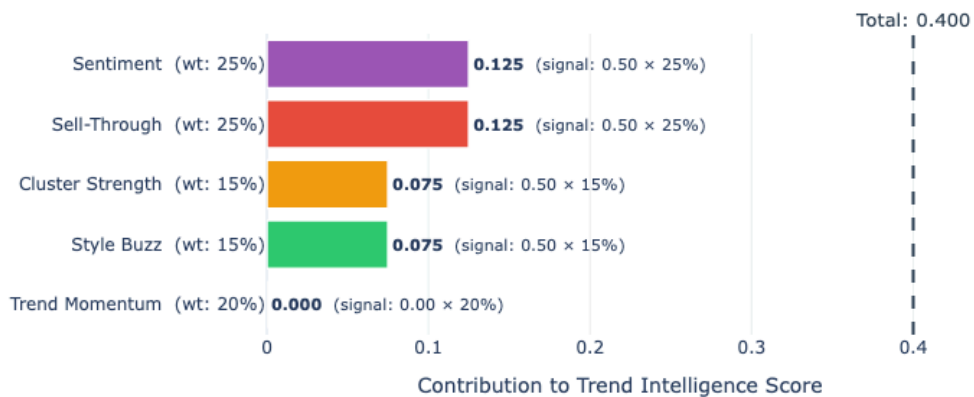
Score Decomposition — Other



Score Decomposition — Shorts

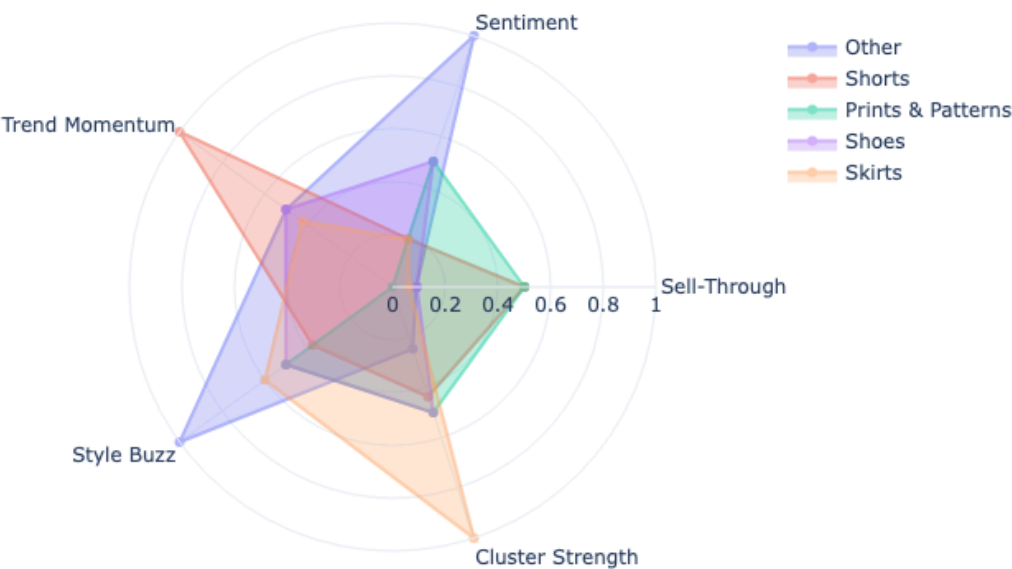


Score Decomposition — Prints & Patterns



8.7 Signal Radar

Signal Radar — Top 5 Categories



9. Key Findings & Recommendations

Executive Summary

The Trend Intelligence Pipeline analyzed 31M+ transactions across 105K products and processed 23K+ customer reviews through three ML models to produce the following actionable intelligence:

HOT : Over-Index Aggressively

Categories in this quadrant have both strong sell-through velocity AND rising social signals. These are validated trends where increased production allocation carries the lowest risk. The specific categories identified depend on the data run, but the quadrant scatter plot in the dashboard provides the definitive view.

EMERGING: Test & Invest

These categories show high social buzz (strong sentiment, trending style tribes, Google Trends momentum) but sell-through hasn't caught up yet. These represent the highest-potential opportunity to get ahead of the curve with small production runs.

Key Methodological Insight

The fine-tuned RoBERTa model captures fashion-domain sentiment that generic tools miss. The zero-shot style classification creates a trend dimension that doesn't exist in the raw data. Together with semantic clustering, the social signal is far richer than simple keyword matching or rule-based NLP.

10. Methodology Transparency

Each category's Trend Intelligence Score can be decomposed into its five contributing signals via the waterfall chart in the dashboard. This transparency allows stakeholders to:

- Understand WHY a category received its score
- Challenge specific signals (e.g., 'Google Trends momentum seems high: is that seasonal?')
- Adjust weights based on business priorities (e.g., 'We trust sell-through more than social buzz')
- Track how signals change over time as the system is updated

Reproducibility

- All parameters, weights, and thresholds are centralized in `src/config.py`
- Random seeds are set for all stochastic operations
- `requirements.txt` pins all dependency versions
- Each notebook documents its assumptions in Markdown cells

11. Assumptions & Limitations

#	Assumption / Limitation	Impact	Mitigation
1	H&M data is a PROXY for the brand's own sell-through data	Category mappings are approximate	Plug in actual brand data when available
2	Customer reviews are a PROXY for social media signals (Instagram, TikTok)	Missing visual/video signals	Phase 1 upgrade adds live Instagram scraping
3	Taxonomy mapping between H&M products and review categories is approximate	Some categories may not align perfectly	A production system would use a trained entity resolver
4	Scoring weights (25/25/20/15/15) are manually set	May not reflect optimal business priorities	Phase 2 adds auto-tuning via backtesting
5	Google Trends may be rate-limited	Missing data for some keywords	Pipeline handles missing data gracefully with defaults

12. Next Steps

The current MVP is a static prototype. The improvement roadmap outlines three phases to transform it into a continuously learning system:

Phase	Focus	Key Deliverable
Phase 1	Live Data Pipeline	Replace static CSVs with weekly Instagram, catalog, and Reddit scrapers
Phase 2	Feedback Loop	Walk-forward backtesting + auto-tuning scoring weights
Phase 3	MLOps Infrastructure	MLflow tracking, Evidently drift monitoring, automated retraining via GitHub Actions

See [FashionTrendIntelligence_Improvement_Roadmap.pdf](#)

Appendix: Tech Stack

Component	Libraries
Data Processing	pandas, pyarrow, numpy
ML / NLP	transformers, peft (LoRA), sentence-transformers, scikit-learn
Clustering	hdbscan, umap-learn
Trends	pytrends
Visualization	plotly, matplotlib, seaborn
Configuration	pydantic
Compute	Colab T4 GPU (free tier all models fit in 16GB VRAM)